

OS Assignment-2

OVERALL DESIGN:

- 1) I used a global variable p which says the function that accesses it to verify tasks from number p to p+TaskInc-1.
- 2) I have functions to check individual rows and columns and grids which will execute in thread according to the task number .
- 3) I intuitively derived the task to be executed from task number by finding (task_num/n) for row /column/grid with values 0/1/2 respectively.
And for finding the entity number (task_num%n).
- 4) The thread goes on to check taskInc number of tasks.
- 5) For string storing I used mutex which is semaphore type implementation which gives access to only one operation at a time in region of m.lock() and m.unlock();

MUTUAL EXCLUSION METHODS

1)TAS Mutual exclusion:

I initialize a global variable u initialised to 1 that will be shared and which acts as lock for a critical section ,we exchange the u with 0 until it gives its oldvalue of 1 to enter critical section ,here I used a different mechanism that is not used in general convention.

```
while(!u.exchange(0,memory_order_acquire));
```

2)CAS Mutual exclusion :

I initialized a global variable the same as before,but here used a different atomic function (that significantly) decreases the cache buffers.Checks whether the global variable is 1 or not if 1 we put its value to 0 and return 1 (or) return 0.

```
int i1=1;
while(!u.compare_exchange_weak(i1,0)) i1=1;
```

3)Bounded CAS Mutual exclusion:

Here we are not going to leave the scheduling CPU, rather we give

access to the critical section according to a scheduled way. In order to make threads get chance more often among themselves.

We define a global array

`int flag[k]={0};` //x->num represents thread number to navigate its flag

```
do{
flag[x->num-1]=1;
    turn =1;
    while(turn &&flag[x->num-1]){
        turn=u.exchange(1,memory_order_acquire);
    }
if(j==i)u.store(0,memory_order_release);
    else flag[j]=0;
    nav=p;

    //critical ends
}while(p<3*n&&!termi);
```

This mechanism gives a circular access to the process waiting for critical section access by making `flag[j]=0`;

Which makes it enter the critical section.

Analysis of graphs:

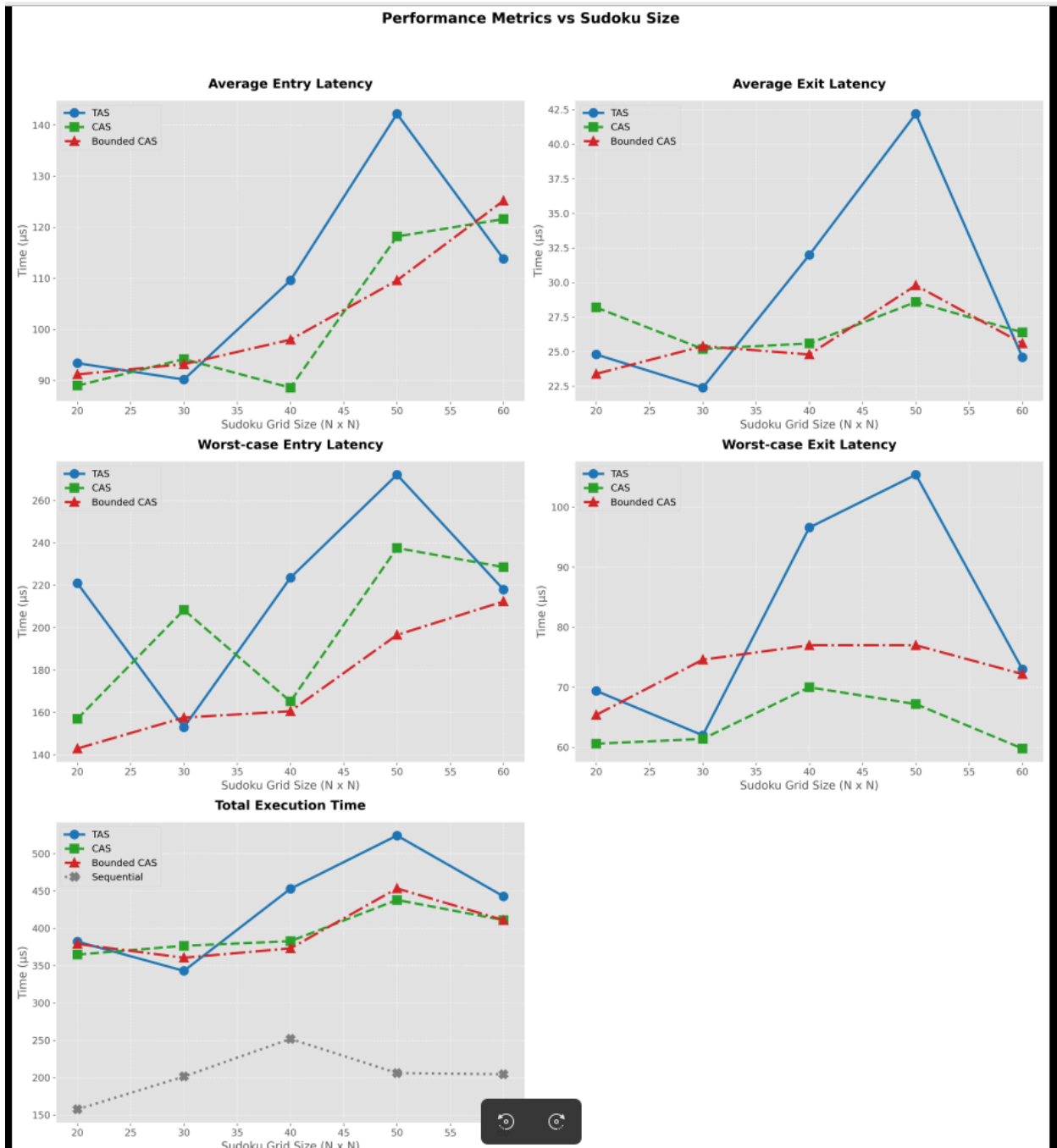
1)SIZE OF ARRAY GETTING INCREASED

- For total time:

1)TAS is showing more overall time ,this is because in presence of common variables, if change is made a multiprocessor caches create more traffic delaying memory due to restriction of maintaining cache coherence.This is the reason for TAS being replaced more by CAS .

2)Sequential is the lowest in the graph since it has the least number of waiting and swaps among other processes is least.

3)Bounded CAS and CAS are almost the same since the core mechanism is the same.



- For time entry time and exit time :
 1) As discussed TAS has obviously more wait times since execution takes more time than others.

2) Bounded CAS has more instruction than CAS in the critical section (since it has to schedule) so exit time on average will be more .

There need not be any increase in general since taskInc is the same and threads are the same.

3) Additional thing to notice here is that worst case time is near to avg. time in bounded CAS which is the main improvisation aimed in Bounded CAS.

2) INCREMENT OF TASKINC

-> Total time :

1) TAS is showing more overall time , this is because in presence of common variables, if change is made a multiprocessor caches create more traffic delaying memory due to restriction of maintaining cache coherence. This is the reason for TAS being replaced more by CAS .

2) Sequential is the lowest in the graph since it has the least number of waiting and swaps among other processes is least.

3) Bounded CAS and CAS are almost the same since the core mechanism is the same.

-> Critical section entry and exit time:

1) As discussed TAS has obviously more wait times since execution takes more time than others.

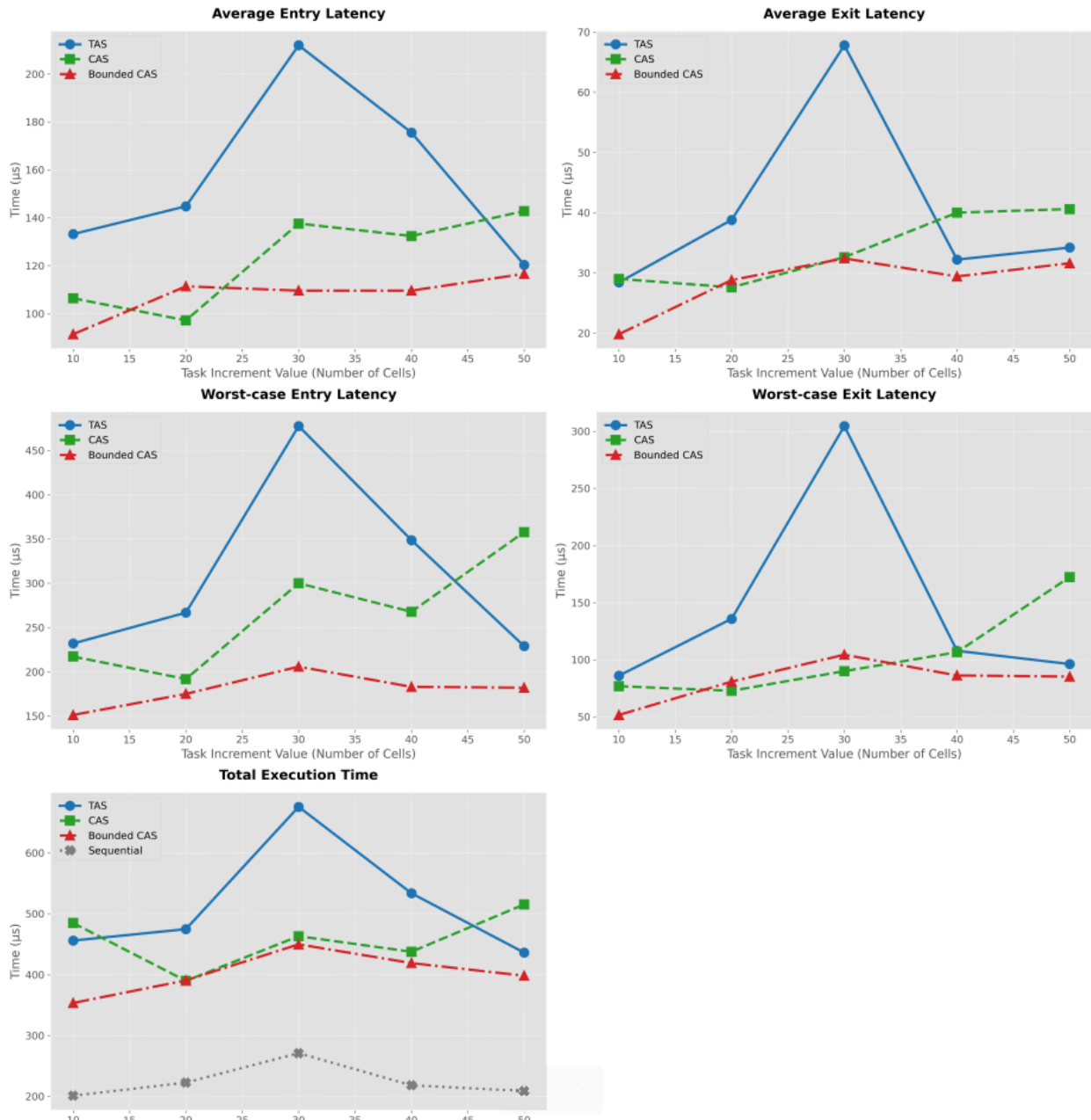
2) Increase in TaskInc doesn't alter the critical section time in general.

3) But an interesting event takes place as taskInc increases the competition for shared variable access , which makes critical section entry time decrease after a number of taskInc.

4) This is why we see a peak coming.

5) Bounded CAS and CAS are almost the same since the core mechanism is the same.

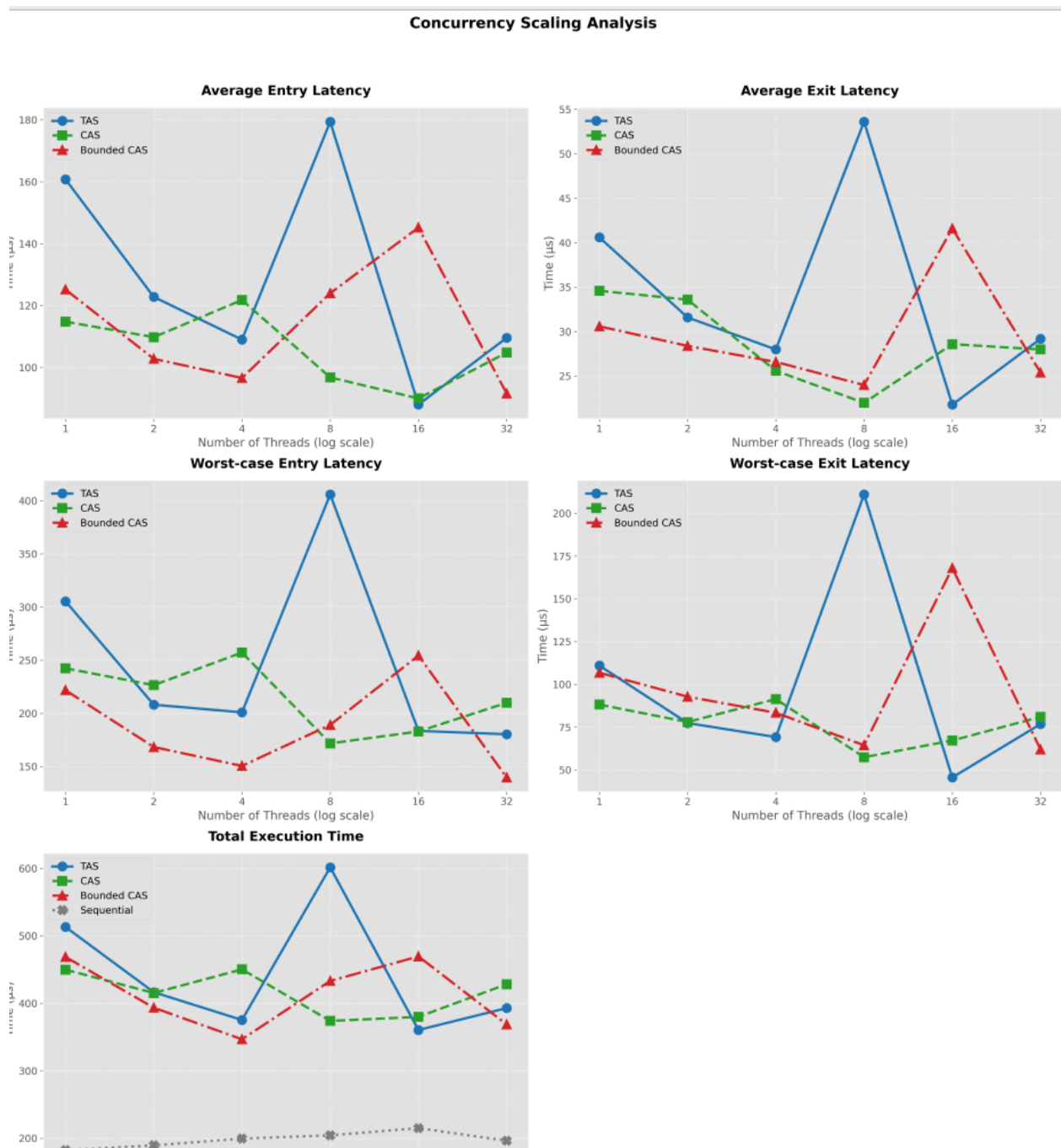
Performance Metrics vs Task Increment Value



3) INCREASING NUMBER OF THREADS IN POOL: FOR TOTAL TIME:

1) TAS is showing more overall time, this is because in presence of common variables, if change is made a multiprocessor caches create more traffic delaying memory due to restriction of maintaining cache coherence. This is the reason for TAS being replaced more by CAS.

2) Sequential is the lowest in the graph since it has the least number of waiting and swaps among other processes is least.



FOR EXIT AND ENTRY TIMES:

1) As the number of threads increases, TaskInc decreases, making competition increase.

2)As competition increases more time the thread has to wait for requests.

3)The decrease we see in the 16 number may be due to perfect synchronisation with the number of cores.

TABLES:

Table 3: Performance Metrics vs Sudoku Size

Grid Size	TAS Avg Entry (µs)	CAS Avg Entry (µs)	Bounded CAS Avg Entry (µs)	TAS Avg Exit (µs)	CAS Avg Exit (µs)	Bounded CAS Avg Exit (µs)	TAS Total Time (µs)	CAS Total Time (µs)	Bounded CAS Total Time (µs)	Sequential Time (µs)
20	1600	50	2200	510	10	420	5000	3000	6200	1900
30	150	180	4200	20	30	600	1100	1200	8800	300
40	250	250	500	120	20	75	1800	1800	2000	300
50	1000	400	1000	330	120	300	3700	2200	3700	900
60	450	70	1700	120	10	230	2600	800	4700	600

Table 1: Concurrency Scaling Analysis

Threads	TAS Avg Entry (µs)	CAS Avg Entry (µs)	Bounded CAS Avg Entry (µs)	TAS Avg Exit (µs)	CAS Avg Exit (µs)	Bounded CAS Avg Exit (µs)	TAS Total Time (µs)	CAS Total Time (µs)	Bounded CAS Total Time (µs)	Sequential Time (µs)
1	150	800	400	10	250	10	1500	2200	2500	700
2	700	600	550	20	180	30	3400	2800	2700	1000
4	720	550	820	220	130	10	3200	2700	3000	700
8	450	1100	2150	10	400	230	2600	3600	5500	800
16	2200	980	150	620	350	10	7800	5400	3500	2000
32	350	550	1200	100	200	120	2200	2800	4100	800

Table 2: Performance Metrics vs Task Increment Value

Task Size	TAS Avg Entry (µs)	CAS Avg Entry (µs)	Bounded CAS Avg Entry (µs)	TAS Avg Exit (µs)	CAS Avg Exit (µs)	Bounded CAS Avg Exit (µs)	TAS Total Time (µs)	CAS Total Time (µs)	Bounded CAS Total Time (µs)	Sequential Time (µs)
10	80	200	500	20	20	150	850	1450	2200	600
20	950	170	100	320	5	50	3300	2300	2300	400
30	280	110	300	30	20	50	1650	2000	1300	1100
40	450	110	1200	30	70	240	2950	2000	3450	600
50	440	220	200	210	10	10	2050	1700	1450	1050